

AI PRODUCT ENGINEER · END-TO-END BUILDER

반복되는 운영 업무를, 프로덕션 AI 시스템으로.

업무의 경계와 예외를 구조화하고,
제품 설계부터 배포·관측·운영까지 책임지는 엔지니어 송희웅입니
다.

620,000+

프로덕션 AI 통화

4,000/일

일 평균 통화

300+

도입 병원

6

다국어 운영

Voice AI에서 AX까지

환자의 전화 업무를 자동화한 경험을 기반으로, 통화 운영
분석과 사내 업무지원 Agent까지 문제의 범위를 확장해
왔습니다.

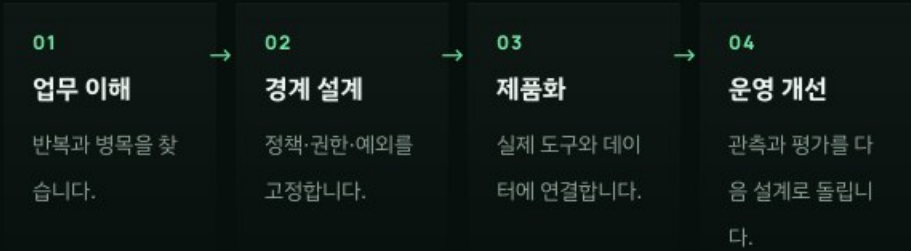
제품을 만드는 방식

9년+ · FULL-STACK TO AI PRODUCT

WORKING THESIS

기술을 먼저 고르기보다 누가 어떤 업무를 반복하고, 어디에서 판단이 흔들리며, 어떤 예외가 운영을 멈추게 하는지부터 정의합니다.

프론트엔드·백엔드·인프라 경험 위에서 Multi-Agent, 실시간 음성 처리, 검색, LLM 평가를 하나의 제품 경계로 연결해 왔습니다. PoC보다 운영 가능한 상태와 다음 개선 근거를 만드는 데 집중합니다.



PRODUCT BUILDING TOOLKIT

OpenAI Realtime API

LiveKit Agents

LangGraph

LangChain

Deep Agents

Claude API

GPT-4.1

Gemini 2.5 Pro

Python

FastAPI

Next.js

React

TypeScript

Qdrant

PostgreSQL

RabbitMQ

Kafka

AWS ECS Fargate

Docker

Terraform

SIP Trunk

Bun

AG-UI

Google Cloud OKF

OPEN SOURCE

LangChain-OpenTutorial

Core Contributor · 2025.02

- 2025.03 - 현재
와이즈에이아이 · AI 프로젝트 팀장
병원 Voice AI 제품과 사내 업무지원 Agent 설계·개발·운영
- 2019.03 - 2025.02
투썸월드 · 소프트웨어 엔지니어
교육/AI 서비스 웹 제품 개발
- 2017 - 2018
투미유 · iOS & Backend 개발
영어회화 학습 서비스 개발

— CAREER JOURNEY · 2025.03 – 2026.08

다섯 프로젝트, 하나의 반복되는 패턴

범위의 확장

전화 한 통의 자동화에서 시작해, 병원별 정책·복합 상담·운영 지표·사내 지식 조회로 문제 범위를 넓혔습니다. 대상은 달라졌지만 매번 사람의 반복 판단을 구조화된 시스템으로 옮겼습니다.



이 포트폴리오가 보여주는 것: Voice AI라는 한 기술이 아니라, 업무를 이해하고 경계를 설계한 뒤 실제 운영 데이터로 개선하는 재사용 가능한 제품 개발 방식입니다.

— PORTFOLIO MAP · CLICK TO JUMP

다섯 Chapter를 한눈에

목차 · 읽기 지도

각 Chapter는 하나의 운영 문제와 검증 가능한 설계 증거를 다룹니다. 행을 누르면 해당 Chapter의 첫 슬라이드로 이동합니다.

01	아웃바운드 Voice AI CALL EXECUTION	voxBridge 수명주기 · SingleAgent · 안전한 예약 신청 · AMD · Kafka 분석 · 이중 배포	7 slides →
02	인바운드 Voice AI POLICY AS DATA	5종 flow_config · Dynamic Booking v3 · 공용 Redis FIFO Warm Transfer · 4단계 검증	4 slides →
03	병원 고객상담 Agent STATEFUL SERVICE	상태형 상담 기반 · 개인정보 수집 후 질문 복귀 · Qdrant 지식 · 현재 Harness 진화	3 slides →
04	통화 관측·평가 OBSERVE THE WORK	결정론적 업무 결과 · 근거 발화 Semantic QA · 개인정보 저장 경계 · 운영 대시보드	2 slides →
05	AIU / AX 업무지원 Agent INTERNAL OPERATIONS	Web·Slack 동일 런타임 · 4 specialist · 5 OKF source · SELECT-only 분석 · OpenBot 실행 경계	7 slides →



— CHAPTER 01 · CALL EXECUTION

병원 아웃바운드 Voice AI Agent

1인 설계 · 개발 · 운영

— BACKGROUND

사람이 반복하던 예약 확인·안내 전화를 병원별 정책과 예외를 유지한 채 실제 전화 망에서 자동화해야 했습니다. 발신 수명주기부터 예약 신청·상담원 연결·분석까지 하나의 제품으로 묶었습니다.

— PROCESS

01 · OWN

voxBridge가 Room·SIP 발신과 cleanup을 단독 소유

02 · CONTROL

SingleAgent·5종 flow node·AgentTask로 대화와 업무 경계 분리

03 · LEARN

Transcript·Event·Recording을 Kafka 분석으로 비동기 전환

— OUTCOME

550,000+

아웃바운드 통화 성공

2,500/일

일 평균 실시간 통화

300+

도입 병원

My Role

Architecture · Core Implementation · Production Operations

PROBLEM

반복 수동 전화와 결과 기록

대상 목록

예약·안내 업무



병원 직원

직접 발신·응대



수기 결과

후속 조치 기록

처리량·일관성·분석 가능성이 사람의 시간과 숙련도에 종속됩니다.

DESIGNED SYSTEM

AFTER

실행부터 분석까지 하나의 운영 시스템

Campaign API

목적·병원 정책



voxBridge

Room · SIP lifecycle



LiveKit

SIP · AMD · Egress



SingleAgent

Flow · Task · Transfer

EXECUTE

예약 신청 · FAQ · Warm/Cold Transfer

OBSERVE

Kafka → 전사 보완 · 결과 재구성 · QA

SIP active gate

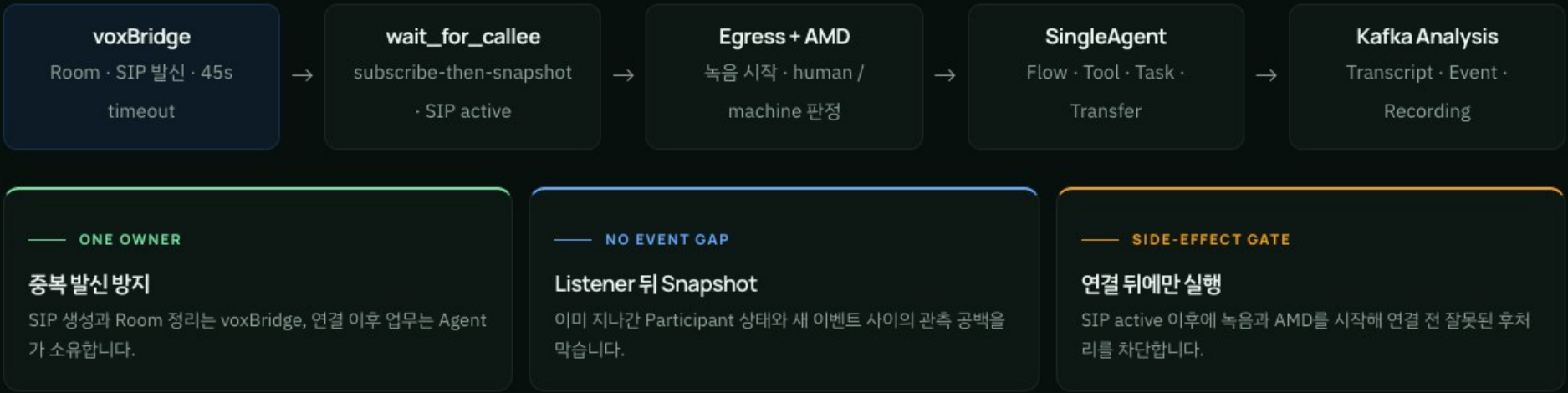
subscribe-then-snapshot

Tokyo ECS · Korea K8s

발신과 대화의 소유권을 분리했습니다

CALL LIFECYCLE

Agent가 직접 전화를 걸지 않습니다. voxBridge가 Room·SIP 발신과 연결 전 실패를 소유하고, Agent는 SIP 상태를 관찰하며 대화·녹음·분석 payload를 책임집니다.



한 Agent가 맥락을, Task가 경계를 소유합니다

SINGLEAGENT · FLOW · TASK

설정이 통화의 큰 흐름을 결정

condition 수신자·병원 설정값으로 분기	
greeting 고정 안내 · Voice / DTMF	agent 업무 대화 · 강제 Tool
action Transfer · Log	exit 종료 안내 · Event
통화 목적도 데이터로 `call_additional_guidance`에 목적·사실 Context·긍정/부정 행동·금지 Tool을 넣어 같은 응답도 통화 목적에 맞게 처리합니다.	

SingleAgent가 유지하는 하나의 Context

CONVERSATION OWNER SingleAgent 예약을 중단하고 병원 정보를 물었다가 다시 돌아와도 같은 세션과 정책 Context를 유지합니다.	
Booking Tools 조회 · 신청 · 변경 요청 · 취소 요청	Bounded Tasks 생년월일 · 진료과 · 일정 · Transfer
FAQ Grounding 시작 시 기관별 활성 FAQ를 주입	Action Mode auto · transfer · leave_memo
중요한 구분: `booking_agent`와 `info_agent`는 별도 Agent가 아니라 설정 호환성을 위한 논리 노드 ID입니다.	

확정 예약이 아니라, 검증된 신청을 접수합니다

SAFE BOOKING · OBSERVABLE VOICE

예약 신청의 결정적 경계

LLM은 대화를 진행하지만, 후보의 존재·허용 범위·최종 동의 일치는 코드와 AgentTask가 검증합니다.

01 · IDENTITY

생년월일 DTMF 수집·낭독 확인

02 · REFERENCE

병원 기준 진료과·의료진 목록 안에서 선택

03 · STAGE

실시간 일정 재조회 후 정확한 후보 스테이징

04 · CONSENT

동의를 한 일시와 스테이징된 일시 일치 검증

05 · SUBMIT

병원 확인 전 예약 신청 정보 기록·안내

정직한 완료 경계: 통화에서 끝나는 것은 예약 확정 이 아니라 신청 접수입니다. 최종 확정 은 병원 확인 절차가 담당합니다.

단계형 Voice Pipeline

단일 Speech-to-Speech 모델보다 업무별 Tool 통제, transcript 감사와 지연 원인 분리가 중요한 환경을 선택했습니다.

AWS STT

Streaming ·
backchannel filter

LLM

GPT-5.4-mini ·
Gemini fallback

Google TTS

Streaming · 병원별
Voice/Speed

Preemptive Generation

Turn 확정 전 LLM과 TTS 일부를 겹쳐 체감 대기
를 줄임

Provider Fallback

Primary 장애가 통화 전체 실패로 번지지 않게 분리

Turn별 관측 지표

STT delay

Turn detection

LLM TTFT

TTS TTFB

Playback

E2E

프로덕션 예외 처리

운영

모델의 대화 능력보다 먼저, 잘못된 순간에 말하거나 연결하거나 종료하지 않도록 상태 경계를 코드로 고정했습니다.



AMD와 첫 발화 경계

FakeAgent가 사람/기계 판정 전 발화를 차단

- 1 SIP active 뒤 LiveKit AMD 실행
- 2 human / uncertain만 SingleAgent 활성화
- 3 Greeting Task에서도 StopResponse 강제

AMD 중 시작된 turn이 Agent 전환 뒤 완료되는 race를 양쪽 실행 주체에서 방어



사용자 무응답 처리

LiveKit `user_state_changed` 활용

- 1 사용자 away 상태 감지
- 2 2회 확인 시도 (각 10초 간격)
- 3 미응답 시 통화 종료

AMD · DTMF Task · Warm Transfer 중에는 전역 무응답 처리를 억제



공용 FIFO Warm Transfer

인바운드·아웃바운드가 같은 상담 자원을 안전하게 공유

- Redis ZSET 기반 병원별 FIFO 대기열
- trunk 점유권과 connected occupancy lock
- AI 브리핑 후 상담원 DTMF 수락
- 대기 재확인 · 재시도 · 수신자 이탈
- 번호 없음·연결 실패 시 메모 fallback

구조화 Event 계약 · 런타임 사실을 문자열이 아닌 typed data로

name

BOOKING_CREATE
WARM_TRANSFER_CALL
CALL_TERMINATION

status

STARTED · STAGED · SUCCESS
FAILED · EXITED

details

reason · stage · mode · count · business_hours · wait_seconds

통화를 처리한 뒤, 운영이 시작됩니다

KAFKA · EVENT · SEMANTIC

실시간 통화는 종료 이벤트와 원본 증거만 Kafka로 전달합니다. 분석 Consumer가 녹음·대화·구조화 이벤트를 결합해 결과를 재구성하므로, 분석 장애가 통화에 영향을 주지 않습니다.

01 · PUBLISH

KAFKA

session · transcript · event · recording anchor

02 · RESTORE

GEMINI 2.5 PRO

상담원 연결 이후 녹음 구간 보완 전사

03 · CLASSIFY

EVENT + TRUSTCALL

결정론 26개 + 의미 3개 Boolean 결과

04 · SUMMARIZE

GPT-5.6 LUNA

Gemini 3.5 fallback · 결과 중심 요약

FACTS FROM EVENTS

실제 실행 결과

BOOKING_CREATE · INFO_LOOKUP · WARM_TRANSFER_CALL · LEAVE_MESSAGE · CALL_TERMINATION의 상태 이력으로 완료·이탈·timeout을 재현합니다.

MEANING FROM CONVERSATION

대화에서만 알 수 있는 의미

trustcall은 거절·공정·불명확 응답만 보완합니다. LLM이 구조화 이벤트가 증명한 업무 결과를 다시 쓰지 않게 역할을 분리했습니다.

재처리 가능한 계약: 원본 증거와 파생 결과를 분리해 모델·규칙이 바뀌어도 같은 통화를 다시 분석할 수 있습니다.

두 배포 경로, 하나의 운영 계약

ECS · KUBERNETES · OBSERVABILITY

환경에 맞춘 배포 경로

— TOKYO · AWS ECS FARGATE

시간대·부하 기반 확장

DEV/QA/PROD · ARM64 · Multi-AZ · 프라이빗 서브넷 · 최대 20 Task

— KOREA · KUBERNETES

voxBridge와 같은 운영 경계

dev/qa/prod cluster · ECR 환경 태그 · rollout status 기반 배포 확인

S3

통화 녹음

Kafka

분석 격리

Redis

Transfer 상태

운영을 지키는 여섯 가지 계약

Lifecycle Ownership

발신·Room 정리는 voxBridge, 대화·분석 증거는 Agent가 소유

Deterministic Boundary

인사말·DTMF·예약 동의처럼 되돌리기 어려운 구간은 Task와 코드가 통제

Provider Fallback

Primary LLM 장애를 Gemini fallback으로 흡수해 한 provider를 통화 실패점으로 만들지 않음

Evidence-first Events

Agent 발화가 아니라 구조화 이벤트 상태 이력으로 업무 결과를 판정

Async Isolation

Kafka로 분석 실패·재처리가 실시간 대화에 역류하지 않도록 격리

Regression from Incidents

AMD handoff·DTMF·Warm Transfer 장애를 재현 가능한 테스트로 전환

병원 인바운드 Voice AI Agent

Core Architecture · Booking · Transfer

— BACKGROUND

환자가 먼저 전화하는 환경에서는 운영시간·DTMF·예약·병원 정보·상담원 연결이 한 통화에 섞입니다. 병원별 차이를 코드 분기가 아니라 검증 가능한 정책 데이터로 바꿔야 했습니다.

— PROCESS

01 · CONFIGURE

5종 flow node로 운영시간·인사·업무·연결·종료 조합

02 · COMMIT

Dynamic Booking v3에서 검색·staging·명시적 동의 분리

03 · SHARE

인·아웃바운드 공용 Redis FIFO와 trunk 점유 상태 관리

— OUTCOME

70,000+

인바운드 통화 성공

1,500/일

일 평균 실시간 통화

4단계

Unit → Text → Sim → SIP 검증

My Role

Runtime Architecture · Dynamic Booking · Warm Transfer · Operations

PROBLEM

병원별 예외가 통화 경로마다 분산

환자 전화

예약·정보·연결

→

병원별 규칙

운영시간·업무 정책

→

개별 처리

예약·상담원·메모

정책 변경이 대화 코드와 섞이면 병원 추가·예외 처리·회귀 검증 비용이 커집니다.

DESIGNED SYSTEM

AFTER

한 Agent 위에 설정·Task·상태 머신을 분리

SIP · LiveKit

Self-hosted runtime

→

flow_config

condition → exit

→

SingleAgent

통화 Context 유지

→

AgentTask

typed result 회수

BOOK 기준정보 → 일정 검색 → staging → 동의

TRANSFER Redis FIFO → trunk → briefing → connected

Client-scoped Context

Stage-before-Commit

ZSET/HASH + Stream

한 Agent와 설정이 병원별 통화를 바꿉니다

SINGLEAGENT · FLOW · CONTEXT

SingleAgent가 통화 전체 Context를 유지하고, `flow\_config`가 운영시간·DTMF·업무 진입·연결·종료를 코드 배포 없이 조합합니다.

condition

병원 설정·운영 상태에 따른 분기

greeting

녹취 고지 · Voice / DTMF

agent

업무 요청 전달 · Tool 진입

action

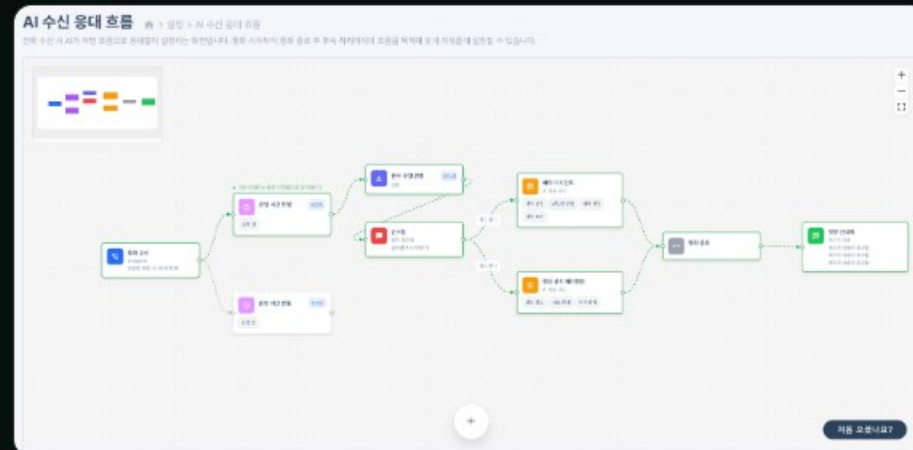
Transfer · Direct · Log

exit

종료 안내 · 구조화 Event

Client-scoped Call Context

시작 시 압축된 병원 근거를 주입하고, 조회 실패 시 아는 척하지 않는 no-knowledge 상태로 통화를 계속합니다.



논리 노드 ≠ Agent handoff: `booking\_agent`와 `info\_agent`는 별도 Agent 인스턴스가 아니라 SingleAgent 안의 설정 호환 ID입니다.

검색과 동의를 분리한 Dynamic Booking v3

STAGE-BEFORE-COMMIT

대화는 LLM, 사실과 상태는 코드

환자의 자연어 선호를 병원 기준정보와 실시간 일정의 교차점에서 검증해, 정확한 후보 하나만 동의 대상으로 만듭니다.

01 · RESOLVE

환자 식별 · 선택적 보험 확인

02 · REFERENCE

허용된 진료과·의료진 ID만 주입

03 · SEARCH

기간·요일·시간·의료진 조건으로 일정 조회

04 · STAGE

실시간 재검증 후 유일한 확인 문구 생성

05 · COMMIT

명시적 동의와 STAGING 일치 시 신청 기록

네 가지 안전 불변식

Search ≠ Consent

일정이 보였다는 사실만으로 확인 질문이나 Commit을 허용하지 않습니다.

Exact Stage

진료과·의료진·일시를 함께 고정하고 새 후보가 오면 이전 staging을 덮습니다.

Explicit Confirmation

되물거나 잘못 들은 “네?”를 동의로 처리하지 않고 staging mismatch를 차단합니다.

Request, not EMR Final

통화 안의 Commit은 완전한 예약 신청 정보 기록이며 병원 최종 확정이 아닙니다.

병원별 Action Mode

auto

AI 처리

transfer

사람 연결

leave_memo

메모 접수

상담원 연결은 병원별 공용 상태 머신입니다

REDIS FIFO · TRUNK · REALTIME STATE

한 건의 SIP 연결보다 어려운 것은 여러 환자가 같은 상담원과 trunk를 요청하는 순간입니다. 인바운드·아웃바운드 호출을 하나의 FIFO와 점유권으로 직렬화했습니다.



세 저장 구조의 역할 분리

ZSET · Source of truth

병원별 활성 FIFO 순서와 rank 0 점유권

HASH · Live read model

상태·시각·시도 횟수·브리핑·통화 방향

STREAM · Notification

PII 없이 상태 변경만 알리고 snapshot을 다시 읽게 함

연결 성공만큼 중요한 실패 복구

- AI가 상담원에게 통화 맥락 브리핑
- 상담원 DTMF 1/2로 수락·거절 결정
- 일부 timeout-error-voicemail 최대 3회 재시도
- 120·240·360초에 대기 지속 여부 재확인
- 수신자 이탈·queue timeout-declined를 별도 상태로 기록
- 연결 번호 없음·실패 시 leave-message fallback
- 상담 종료 전까지 connected 호출이 trunk 점유

01 · Unit

규칙·상태·API

02 · Text Eval

LLM 판단

03 · Patient Sim

멀티턴·예외

04 · Audio / SIP

STT·DTMF·전화망

— CHAPTER 03 · STATEFUL SERVICE

병원 고객상담 AI Agent

— BACKGROUND

병원 상담은 FAQ 검색만으로 끝나지 않습니다. 질문 중 개인정보 수집·예약·상담원 연결 절차가 끼어들면 사용자가 같은 맥락을 반복해야 했고, 검색 결과와 상담 상태를 하나의 Thread에서 관리할 기반이 필요했습니다.

— PROCESS

01 · REMEMBER

pending_question·collected_info·current_node를 Graph state로 관리

02 · GROUND

병원 Fact Data를 Qdrant Dense+Sparse Knowledge Tool로 연결

03 · SERVE

Thread 기반 FastAPI·LangGraph API·Docker 서비스 경계 구축

— OUTCOME

질문 복귀

개인정보 절차 후 원래 상담 재개

병원별 지식

기관·의료진·시설·증명서 Tool

확장 기반

Harness · Voice · Kakao 진화 수용

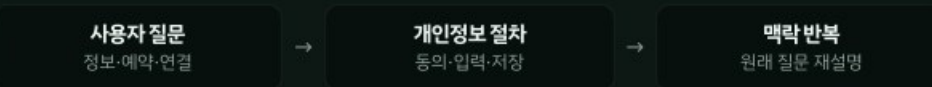
My Role

Initial Graph · State/Tool Flow · Knowledge · API Boundary

Foundation Architecture · 2025.03-2026.03

PROBLEM

절차 전환 때 상담 맥락이 끊김



상담·개인정보·검색 결과가 분리되면 대화 경험과 후속 실행 모두 불안정해집니다.

DESIGNED SYSTEM

AFTER

질문·절차·근거를 공유하는 Stateful Graph



COLLECT 동의 → 추출 → 검증 → 저장 → 질문 복귀

GROUND Qdrant Hybrid Search → typed Knowledge Tool

Postgres · Redis

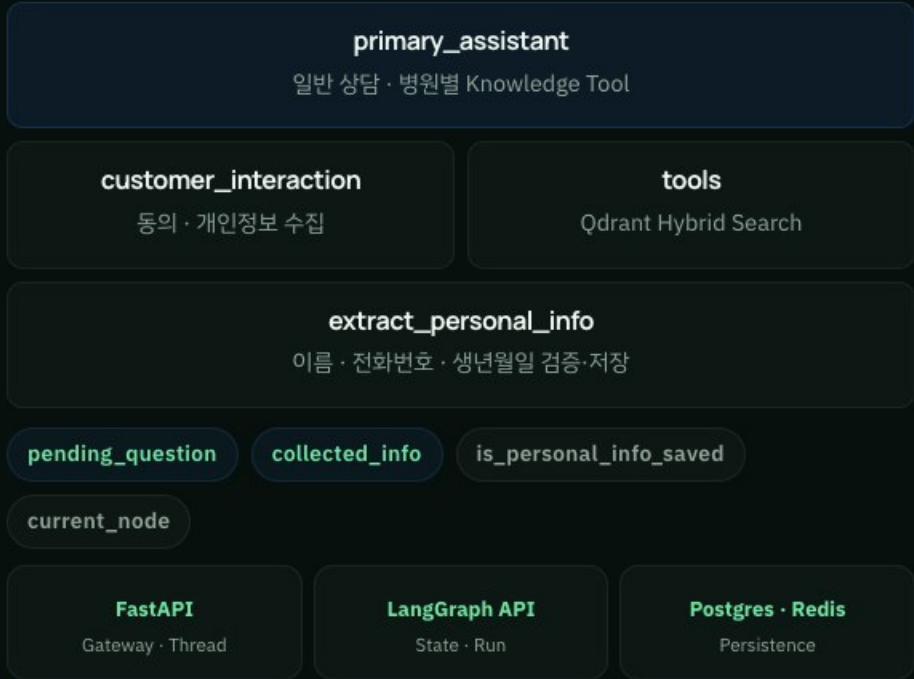
Docker service boundary

Team-added Harness guard

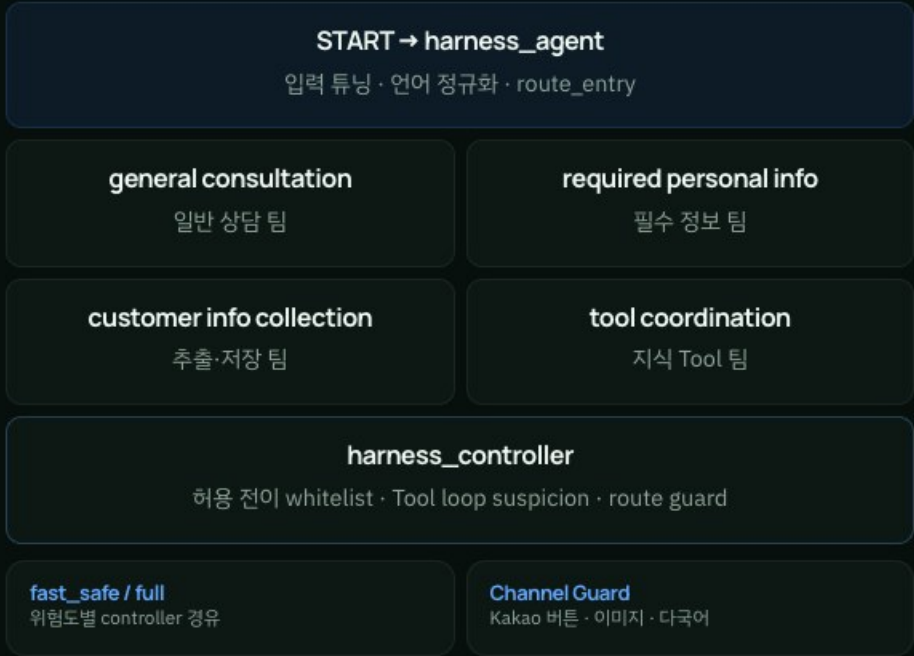
상태형 상담 기반이 Harness Runtime으로 진화했습니다

CONTRIBUTION
BOUNDARY

내가 설계·구현한 기반



이후 팀이 확장한 현재 Runtime



확장 가능성을 증명한 지점: 현재 Graph는 초기 상담 노드를 team wrapper로 감싸고 `current\_node` legacy 값을 유지합니다. 기반을 버리지 않고 통제 계층을 추가했습니다.

상담 절차가 끼어들어도, 대화의 목적은 사라지지 않습니다

LIVE DEV SERVICE ·
SYNTHETIC DATA

진료시간 질문과 상담원 연결이 한 문장에 섞인 요청을 실제 개발 서비스에서 검증했습니다. 먼저 답할 수 있는 정보는 제공하고, 연결에 필요한 정보만 수집한 뒤 같은 Thread에서 신청을 완료합니다.

정보 응답 → 개인정보 수집01 · BRANCH

<와이즈병원 챗봇

진료시간을 안내드립니다.
월, 화, 목, 금: 오전 07:00 ~ 오후 07:30까지입니다 (점심시간은 오후 01:00~ 오후 02:00)
수: 오전 09:00 ~ 오후 06:30 까지이고요 (점심시간은 오후 01:00~ 오후 02:00)
토: 오전 07:00 ~ 오후 02:00 까지입니다.

상담 연계를 위한 개인정보 수집·이용에 동의하는 경우 아래 정보를 입력해 주세요.
개인정보를 입력하지 않아도 챗봇은 이용할 수 있습니다.

ANSWER 진료시간 답변을 먼저 완결하고 이름·전화번호·생년월일만 요청합니다.



동의 확인 → 상담 신청 완료02 · RESUME

<와이즈병원 챗봇

예시:
홍길동 01012345678 19900115
개인정보 처리방침 확인하기

홍길동 01012345678 19900115

RESUME 예시 데이터를 확인한 뒤 질문을 다시 설명하게 하지 않고 연결 신청을 완료합니다.

State · pending question과 수집 정보를 Thread에 유지

Boundary · 개인정보 미입력 상태에서도 일반 상담 가능

Evidence · 공개 개발 환경, 화면의 값은 예시 데이터

Voice AI 통화 관측·평가 파이프라인

Analytics v2 · Privacy-safe LLM Evaluation

— BACKGROUND

통화가 하루 수천 건씩 쌓이자 사람이 녹취를 다시 듣고 결과를 판정하는 일도 반복 업무가 됐습니다. 실시간 통화와 분석 부하를 분리하면서도 한 통화의 업무 결과와 품질을 재구성해야 했습니다.

— PROCESS

01 · DECOUPLE

Transcript-Agent
event-Recording-Laten-
cy를 Kafka로 비동기 전
달

02 · RECONSTRUCT

결정론적 event를 시도와
통화별 최종 resolution
으로 정규화

03 · EVALUATE

근거 turn을 포함한
Semantic QA와 버전 점
수 발행

— OUTCOME

Analytics v2

KPI에서 책임별 손실까지
drill-down

재평가 가능

room-평가 버전 기반 맥등 처
리

Privacy-safe

민감 원문은 로컬, 집계만 외
부

My
Role

Pipeline Architecture · Result Normalization · LLM Evaluation · Privacy
Boundary

PROBLEM

운영자가 통화 원문을 직접 확인

실시간 통화

대화-Tool-Transfer

→

녹취-로그

서로 다른 신호

→

수동 판정

결과-품질 확인

분석 장애가 실시간 경로에 영향을 줄 수 있고, 같은 통화도 평가 기준에 따라 결과가 달라질 수 있습니다.

DESIGNED SYSTEM

AFTER

실행 신호와 의미 평가를 분리한 후처리

Kafka

비동기 event stream

→

Consumer

맥등-재처리

→

Analytics v2

attempt · resolution

FACTS 예약-연결-종료를 구조화 event로 결정

MEANING 의도-라우팅-응답 품질을 근거 turn으로 평가

Gemini 보완 전사

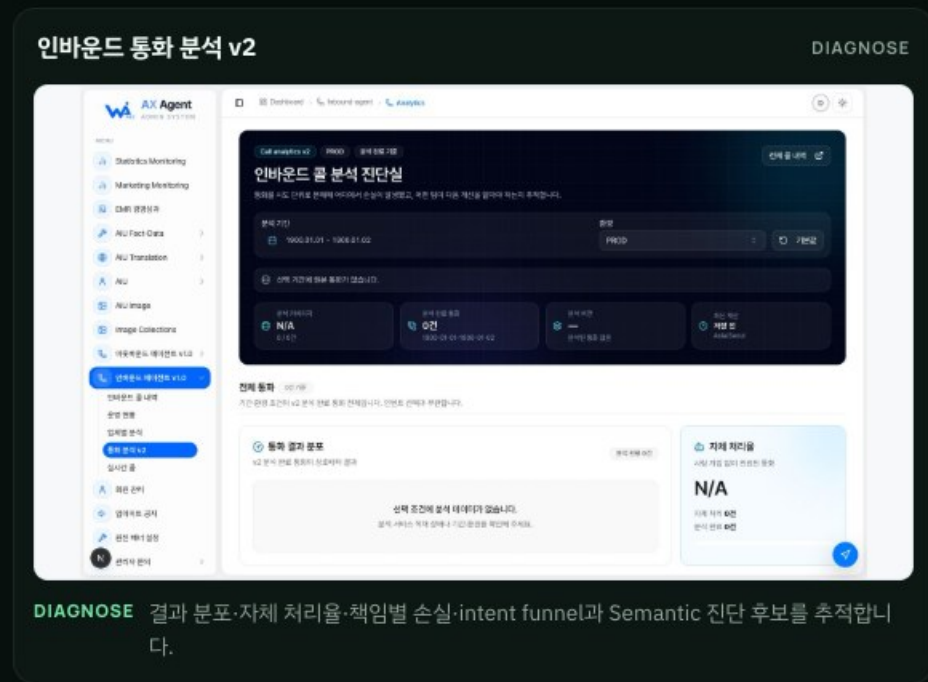
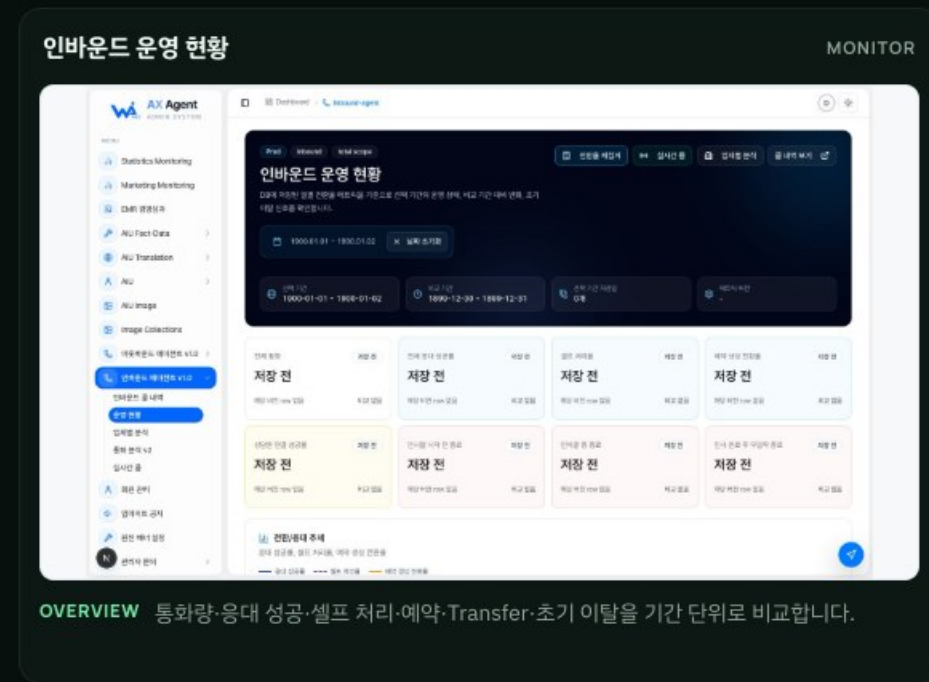
Langfuse versioned score

PII local boundary

운영 성과에서 손실 원인까지, 두 개의 해상도로 봅니다

KPI OVERVIEW · ROOT-CAUSE DRILL-DOWN

운영 현황은 기간별 성과와 추세를 빠르게 확인하고, 통화 분석 v2는 결과 분포와 intent attempt 단위 funnel을 내려가며 손실 지점을 진단합니다.



Facts · 완료 여부는 구조화 Agent event로 재구성

Meaning · 품질 평가는 근거 turn과 평가 버전을 유지

Privacy · 화면은 매뉴얼의 빈 기간 예시이며 현재 운영 수치가 아님

다음 확장: 환자 업무를 실행하고 평가한 경험은, 동료가 운영 정보와 지식을 찾는 반복 업무를 자동화하는 AIU로 이어졌습니다.

사내 업무지원 Multi-Agent

AIU / AX 자동화 · 2026.08~

— BACKGROUND

AIU 제품과 통화 운영 정보를 5개 시스템·매뉴얼·화면에서 찾아야 했습니다. 동료의 반복 조회를 줄이되, 지식 접근·DB 조회·브라우저 실행 권한을 Agent의 추론과 분리해야 했습니다.

— PROCESS

01 · UNIFY

5개 source를 OKF v0.2 catalog와 read-only backend로 통합

02 · DELEGATE

Supervisor가 4개 stateless specialist에 완전한 작업을 위임

03 · GOVERN

AG-UI·SELECT-only DB·OpenBot Tool ownership으로 실행 경계 유지

— OUTCOME

약 3주

1차 운영 가능 상태

322개

Manual · Playbook · Policy 통합

45개

Unit · Contract Test

My Role

Architecture · Migration · Knowledge Contract · Safe Tools · Web/Slack Runtime

PROBLEM

사람이 여러 시스템에서 근거를 조합

업무 질문

제품·통화·화면

→

5개 Source

매뉴얼·운영 화면

→

수동 조합

검색·조회·캡처

정보 위치와 실행 권한이 흩어져 있어 질문마다 화면 탐색과 근거 확인을 반복합니다.

DESIGNED SYSTEM

AFTER

Web·Slack 하나의 접점, 분리된 권한 경계

Web · Slack

사용자의 업무 언어

→

OpenBot

run · policy · audit

→

Supervisor

route · compose

→

4 Specialists

source isolation

KNOWLEDGE OKF catalog → 승인 문서·screenshot

OPERATIONS SELECT-only MySQL → 집계·bounded QA

5 source contracts

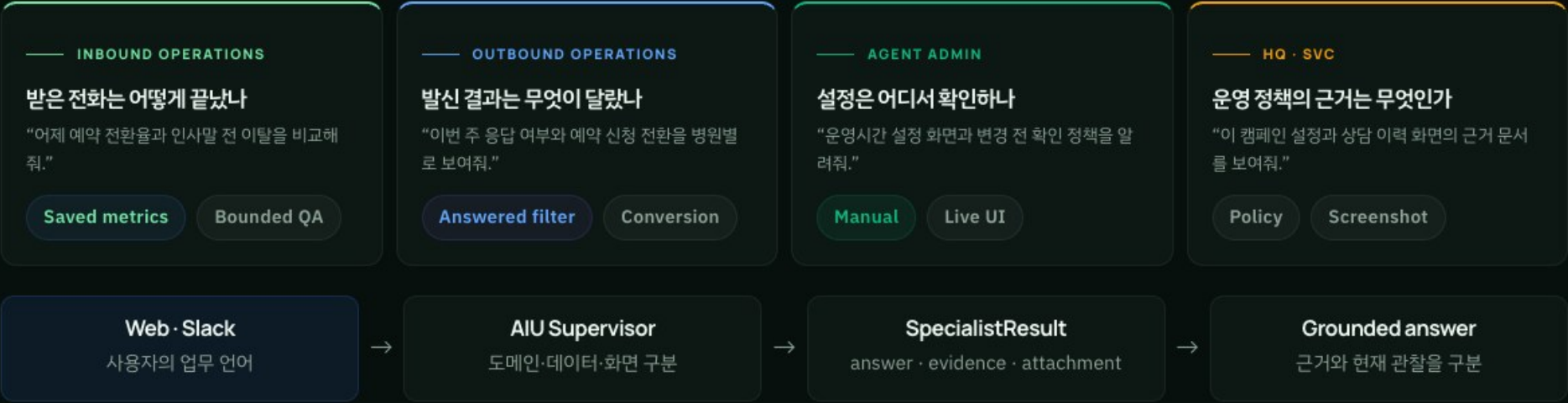
AG-UI final boundary

Short-lived attachment

업무 화면을 찾는 대신, 업무 언어로 묻습니다

WEB · SLACK · ONE RUNTIME

같은 자연어라도 필요한 근거와 실행 경계는 다릅니다. Supervisor가 제품·데이터·화면 요청을 구분하고 해당 specialist에 완전한 작업 설명을 전달합니다.



ONE RUNTIME · TWO SURFACES

“운영 현황과 통화 분석 화면의 차이를 근거와 함께 설명해 달라”는 동일 질문을 Web과 Slack에서 실행했습니다. 두 채널 모두 같은 Agent Admin 지식과 승인된 매뉴얼 screenshot을 사용합니다.

[illegible][illegible]

Surface-owned delivery · Web attachment / Slack file

Supervisor는 답을 만들기보다 경계를 지킵니다

DEEP AGENTS · AG-UI

Supervisor + 4개 stateless specialist

AIU Supervisor
제품·데이터·화면 구분 · 최종 답변 조합

Inbound Agent
저장 통화 · 전환율 · QA

Outbound Agent
응답 결과 · 예약 전환

Agent Admin Agent
매뉴얼 · 화면 · 변경 정책

HQ/SVC Agent
운영 정책 · HQ/SVC UI

실제로 강제되는 source isolation

- Supervisor는 `/knowledge/sources` read가 deny
- Specialist는 선언된 source path와 Tool만 read

AG-UI가 내부 작업과 surface를 분리

OpenBot Run
verified run context · Web / Slack surface

Deep Agent Event Stream
하위 Agent event와 JSON result는 내부에서 처리

Supervisor Final
answer · evidence를 조합하고 attachment metadata를 검증

Surface Boundary
최종 text 또는 OpenBot이 실행할 Tool call만 노출

Surface Tool
호출권을 Web/Slack OpenBot으로 반환

Deployment Tool
active run + callback credential 확인

322개 문서를 한 폴더가 아닌 다섯 계약으로

OKF V0.2 · READ-ONLY BACKEND

각 source는 별도의 snapshot·기본 접근 metadata·문서 수·entrypoint를 갖습니다. Catalog는 모델이 읽기 전에 manifest와 실제 파일을 대조합니다.

43 Inbound Agent runtime · booking · transfer	33 Outbound Agent call flow · operations	62 Agent Admin screen · auth · policy	51 AIU HQ head-office operations	133 AIU SVC hospital-facing UI
--	---	--	---	---

Catalog가 시작 시 검증하는 것

Source Manifest source_id · snapshot · entrypoint	Document Count 선언 수와 실제 Markdown 일치
Frontmatter classification · roles · claims	Screenshot Path 문서가 참조하는 화면 자산

코드로 강제되는 현재 경계

- `/knowledge``는 write가 항상 deny
- Supervisor는 모든 source 원문 read가 deny
- Specialist는 자신의 source path만 read
- Screenshot은 catalog가 승인한 문서·index만 반환

정직한 현재 범위: verified AIU caller는 아직 동일 full-access identity를 사용합니다. Frontmatter role/claim은 계약 metadata이며 사용자별 RBAC 완성으로 표현하지 않습니다.

운영 데이터를 읽되, 조회 범위는 코드로 고정했습니다

SELECT-ONLY · PRIVACY BOUNDARY



일상 운영 조회

Inbound
저장 통화 검색 · 자체 처리율 · 예약 전환 · 상담원 연결 · 일별 운영 metric

Outbound
응답/무응답 filter · 통화 결과 · 예약 신청 전환 · 명시적 기간 집계

이름 제외

전화번호 제외

Transcript 제외

Raw payload 제외

민감 통화 Deep Review

최대 5건
명시적 record id

최대 120 turn
앞·뒤 bounded view

- 이름·전화번호·패턴형 식별자 redaction
- transcript를 untrusted data로 취급
- evidence turn index와 limitation만 유지
- 최종 답변에 raw transcript·개인정보 재노출 금지

Fail-closed by configuration: SELECT-only 계정이 없으면 지식·Computer Use는 계속 동작하지만 DB Tool은 명확한 설정 오류를 반환합니다.

실행 권한과 전달 경계를 OpenBot에 남겼습니다

TOOL OWNERSHIP · ATTACHMENTS · SCOPE

AIU Agent는 reasoning과 위임을 담당하지만 브라우저·plugin·surface Tool의 실행 주체가 아닙니다. OpenBot이 검증된 run, 사용자 surface와 실행 정책을 계속 소유합니다.

ADMISSION

OpenBot Run Assertion

AG-UI 요청에 run context와 `aiu-agent` Bot ID가 없으면 401로 거부합니다.

EXECUTION

Tool Ownership

Surface Tool은 OpenBot UI로 반환하고 deployment Tool은 callback credential로 실행합니다.

COMPUTER USE

Policy · HITL · Audit

브라우저와 plugin 권한, secret 입력과 사람 승인은 OpenBot 정책 경계에 남깁니다.

DELIVERY

Short-lived Attachment

매뉴얼 screenshot은 HMAC URL, 생성 이미지는 난수 URL로 대화 상태 밖에서 전달합니다.

2026.08 착수 · 약 3주 만에 1차 운영 가능한 상태

1차 범위에서 완료

- Deep Agents 마이그레이션 + AG-UI Web·Slack 런타임
- 5 source OKF catalog와 specialist source isolation
- SELECT-only 운영 조희·최대 5건 비식별 QA
- OpenBot Tool ownership·첨부 경계, 45개 AIU test

아직 말하지 않는 것

- 절감 시간·전사 확산률처럼 측정하지 않은 성과
- verified caller별 세밀한 role/claim RBAC
- DB 계정이 없을 때의 운영 분석 실행
- 세일즈·재무·HR 전사 업무 자동화

ONE PATTERN, MANY OPERATIONAL PROBLEMS

업무를 이해하고, 경계를 설계하고, 운영으로 증명합니다.

Voice AI, 통화 지표 자동화, 사내 업무지원 Agent까지.

기술이 달라져도 반복 업무를 프로덕션 AI 시스템으로 바꾸는 방식은 같습니다.

✉ kofsitho@naver.com

🐙 GitHub

🌐 LinkedIn

📝 Tech Blog

📖 Medium

Workflow Structuring

Multi-Agent Systems

Observability & Evaluation

Production Operations

End-to-End Ownership